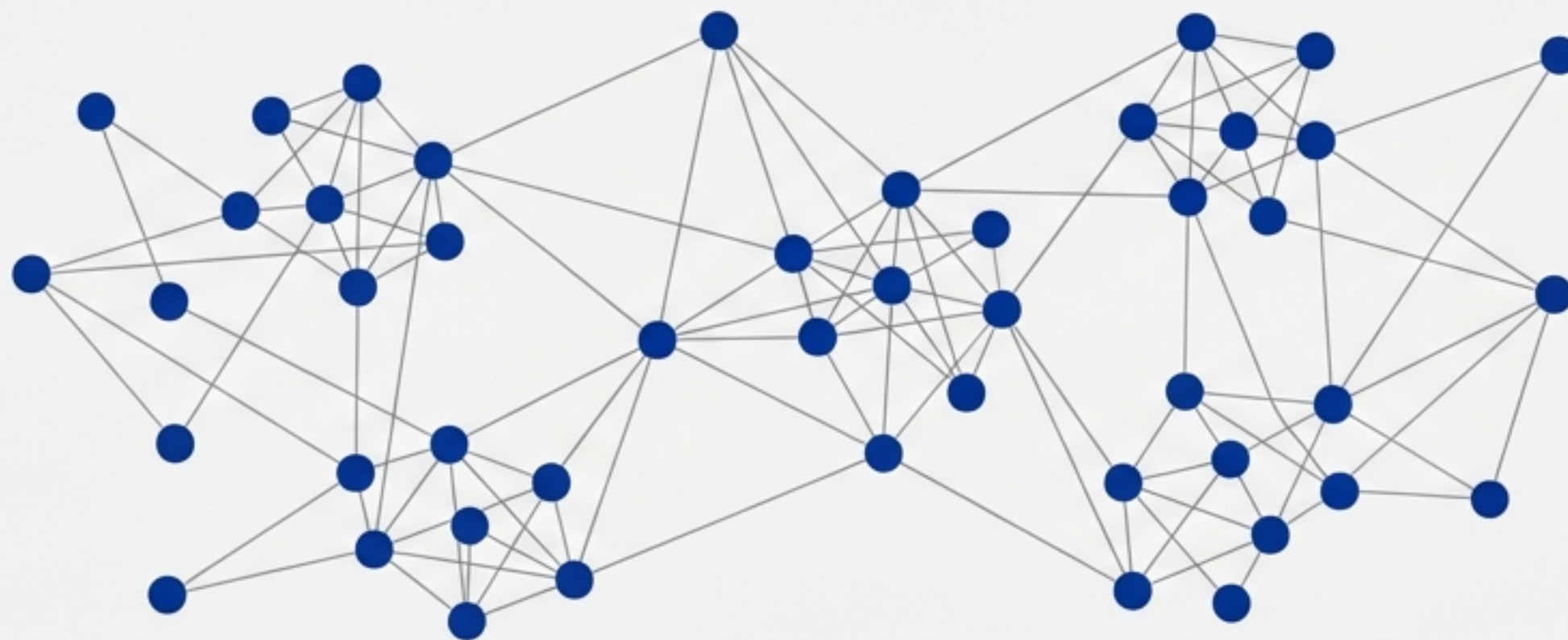


ยุคแห่งองค์กรตัวแทน AI

การเรียนรู้เพื่อจัดระเบียบกระบวนการคิดด้วย Language Models



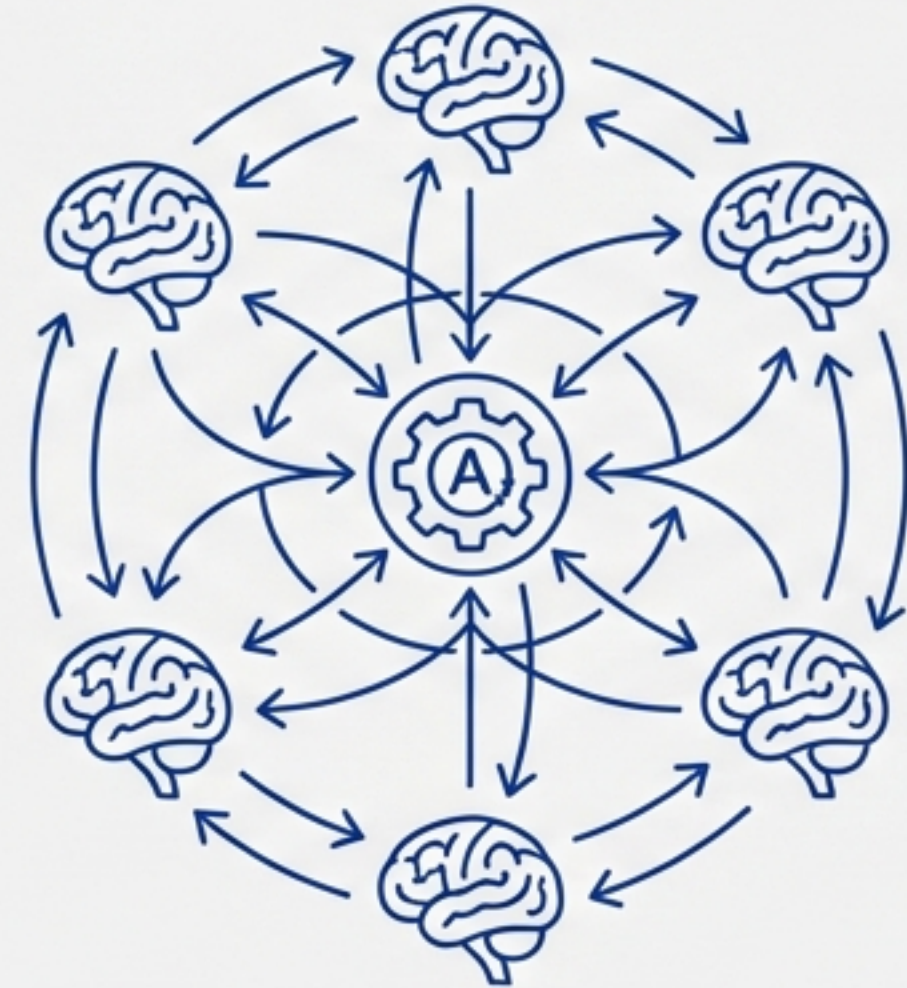
Based on 'The Era of Agentic Organization' by Microsoft Research

วิสัยทัศน์: จาก ‘ความฉลาดรายบุคคล’ สู่ ‘ระบบที่ทำงานร่วมกัน’

สถานะปัจจุบัน



อนาคต: Agentic Organization

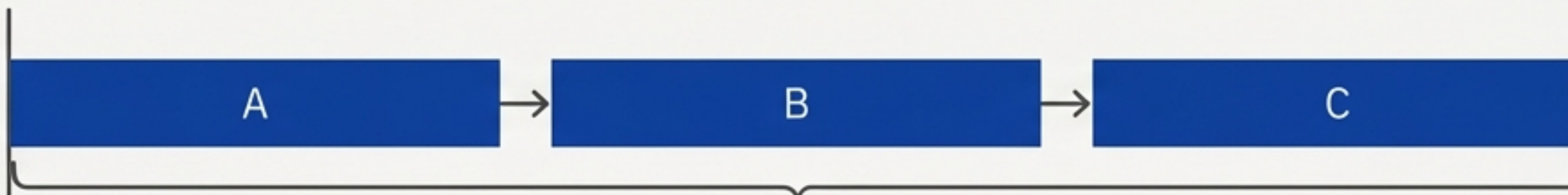


Agentic Organization (องค์กรตัวแทน)

แทนที่จะพึ่งพา AI เพียงตัวเดียว เราสร้างระบบที่ **Agents** หลายตัวทำงานร่วมกัน (**Collaboratively**) และทำงานพร้อมกัน (**Concurrently**) เพื่อแก้ปัญหาที่ซับซ้อนเกินกว่าขีดจำกัดของตัวเดียว

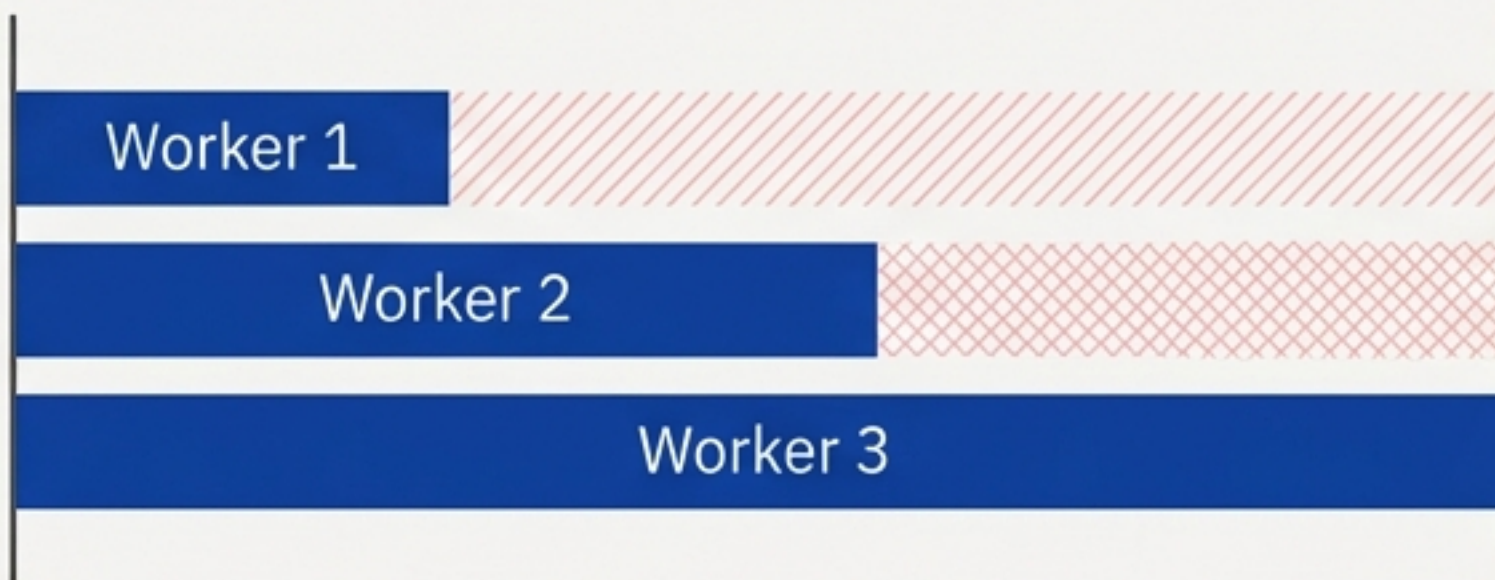
ข้อจำกัดของสถาปัตยกรรมปัจจุบัน

Sequential Thinking
(การคิดแบบลำดับ)



ความหน่วงสูง (High Latency) - ต้องรอให้ขั้นตอน A จบก่อนเริ่ม B

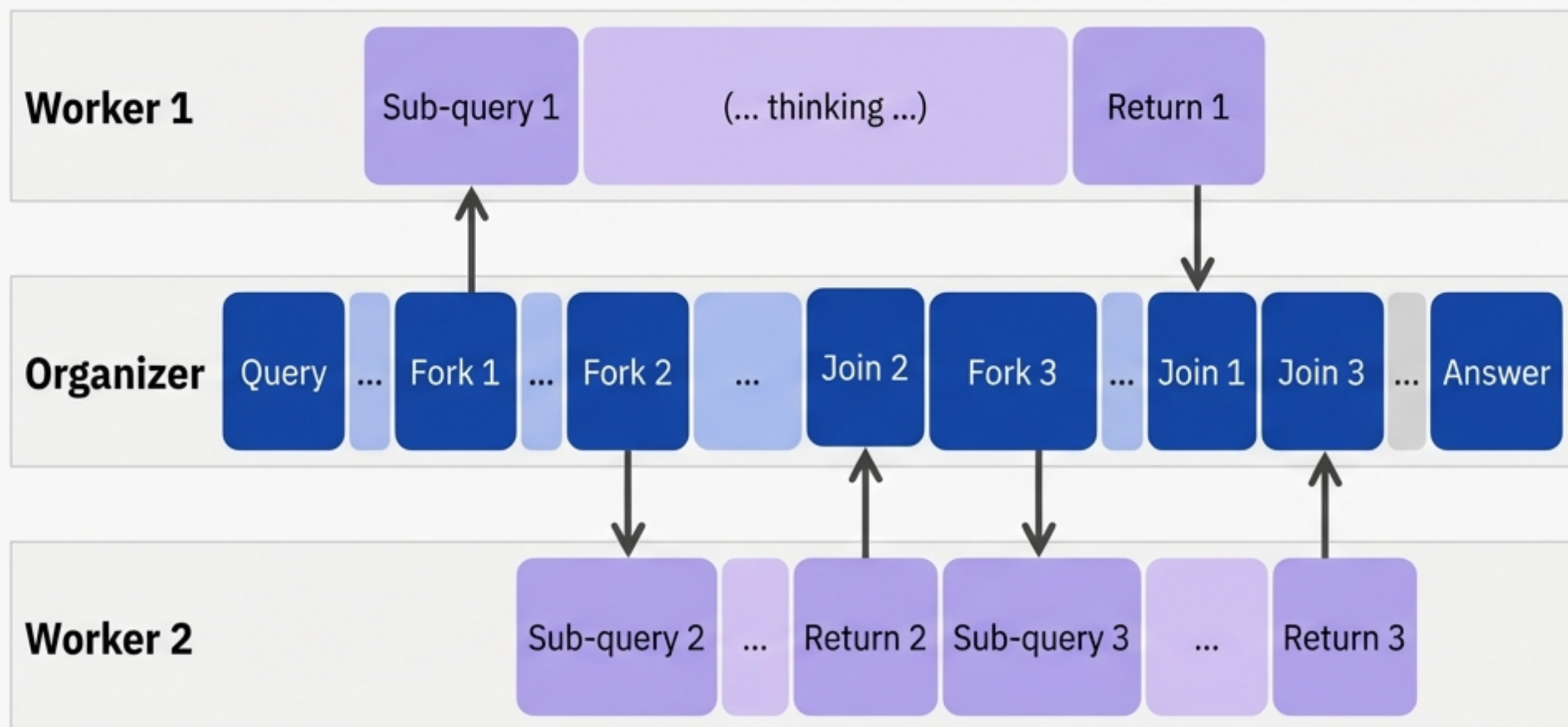
Parallel Thinking
(การคิดแบบขนาน)



โครงสร้างตายตัว (Fixed Structure)
- ถูกจำกัดโดย Worker ที่ช้าที่สุด

Insight: วิธีการปัจจุบันขาดความยืดหยุ่น (Adaptivity) และ
ความมีพลวัต (Dynamism) ทำให้เกิดความล่าช้าในการรวบรวมข้อมูล

ขอแนะนำ AsyncThink: กระบวนทัศน์ใหม่ของการคิด



ลดความหน่วง
(Latency)

ลง **28%**

เพิ่มความแม่นยำ
ในงานที่ซับซ้อน

โมเดลเรียนรู้ที่จะจัดระเบียบกระบวนการคิดภายในให้เป็นโครงสร้างที่ทำงานขนานกันได้
(Concurrently Executable Structures)

AI เปรียบเสมือนระบบคอมพิวเตอร์

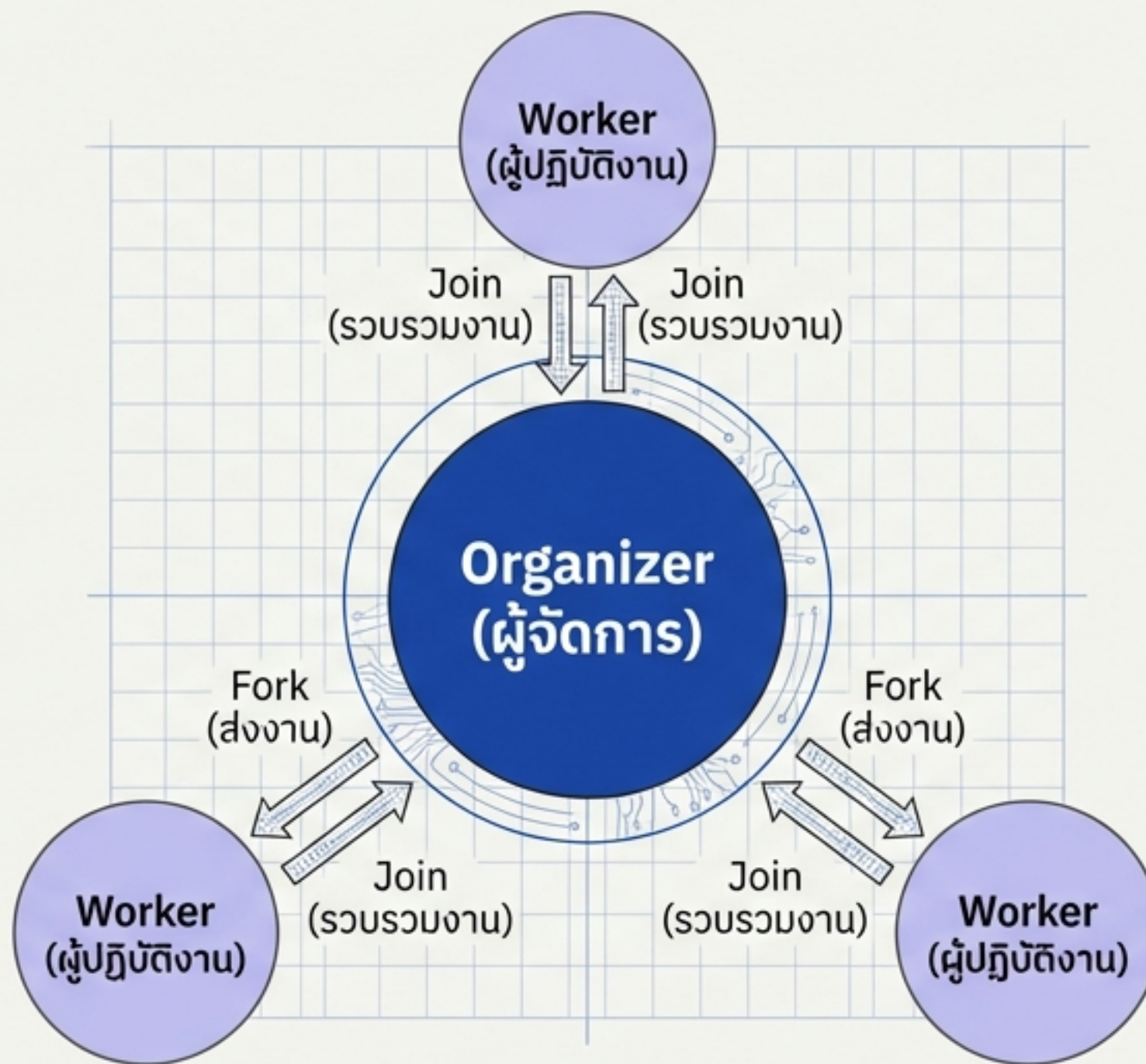
แนวคิด AI	เปรียบเทียบกับระบบคอมพิวเตอร์	รายละเอียด
Agent (ตัวแทน)	CPU Core	หน่วยประมวลผลเดี่ยว
Agent Pool (กลุ่มตัวแทน)	Multi-core CPU	รองรับการทำงานแบบขนาน (Parallel Capacity)
AsyncThink	OS Scheduler	จัดการ Process อย่างชาญฉลาด

AsyncThink คือ ‘Organization Policy’ หรือนโยบายการจัดระเบียบที่เปลี่ยน LLM ให้ทำงานเหมือนโปรแกรม Multiprocess ที่มีประสิทธิภาพสูงสุด

โปรโตคอล: ผู้จัดการและผู้ปฏิบัติงาน

Organizer (ผู้จัดการ)

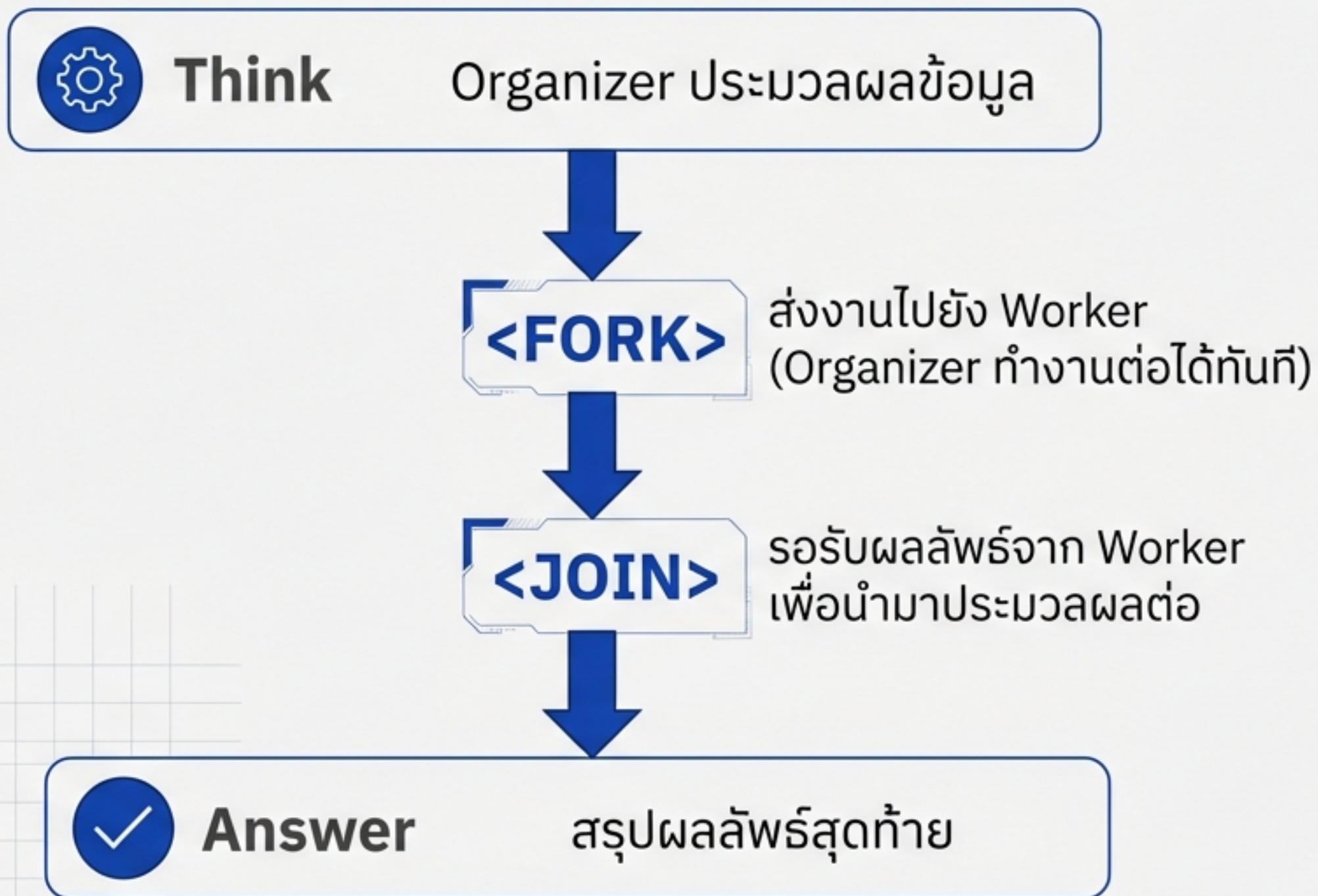
- รับผิดชอบการประสานงานทั่วโลก (Global Coordination)
- ควบคุม Flow: ส่งงาน (Fork) และ รวบรวมงาน (Join)
- Input: Query + System Prompt



Workers (ผู้ปฏิบัติงาน)

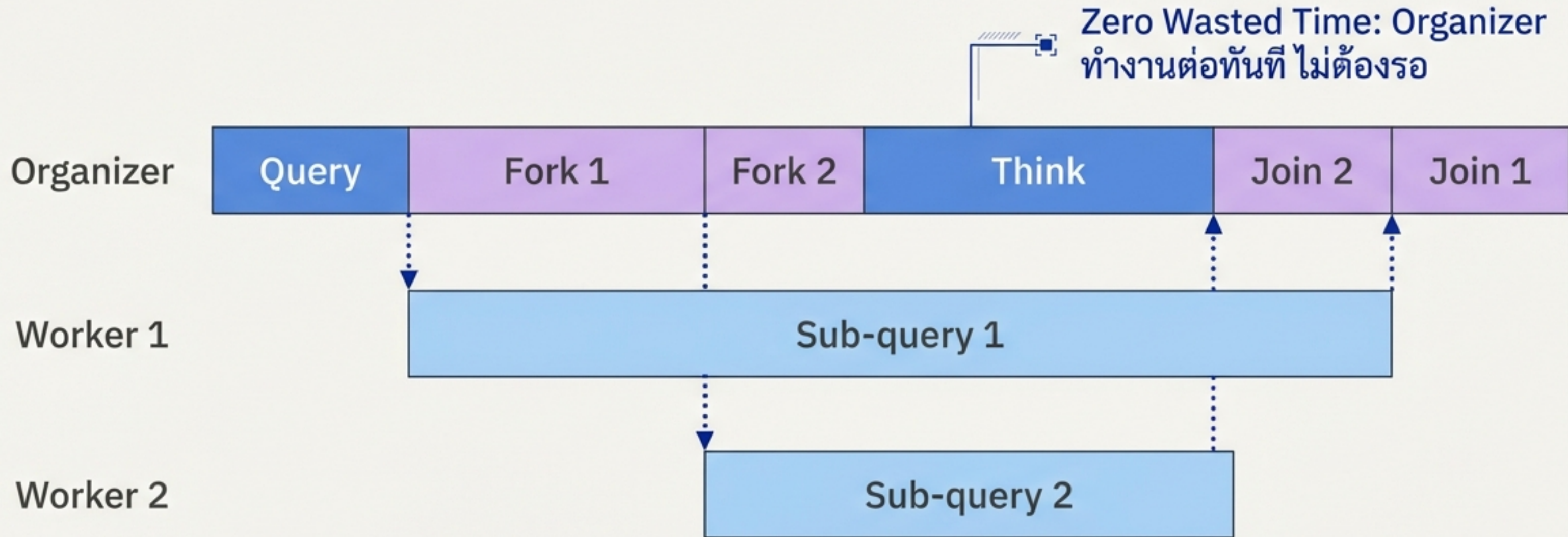
- รับผิดชอบการประมวลผล Sub-query (งานย่อย)
- ทำงานอย่างอิสระและส่งคืนผลลัพธ์
- Input: Sub-query + System Prompt

กลไกการควบคุม: Fork & Join



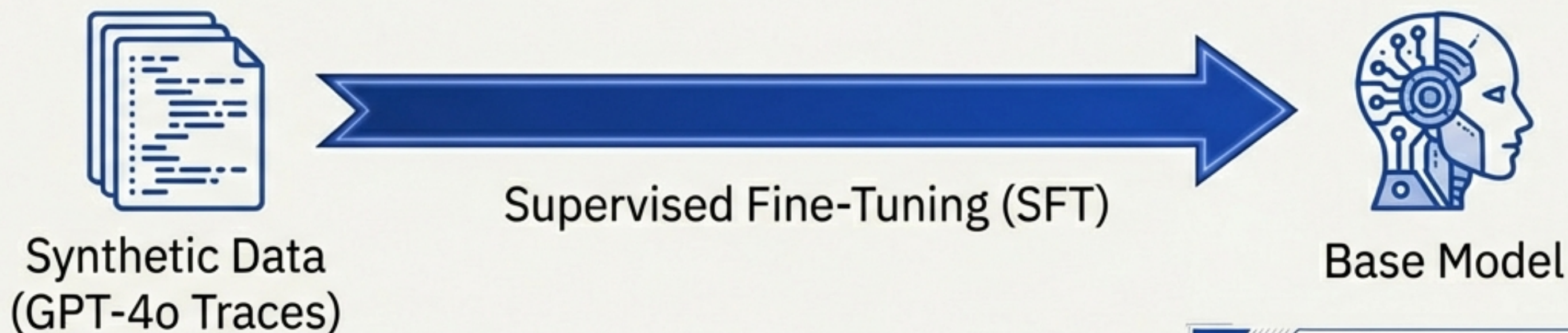
ช่วยให้ LLM สามารถจัดการ 'Context' ของตัวเองได้เหมือนกับการเขียนโปรแกรมแบบ Asynchronous

การทำงานแบบพลวัต (Dynamic Flow in Action)



ขั้นตอนที่ 1: การเรียนรู้รูปแบบคำสั่ง

Cold-Start Format Fine-Tuning

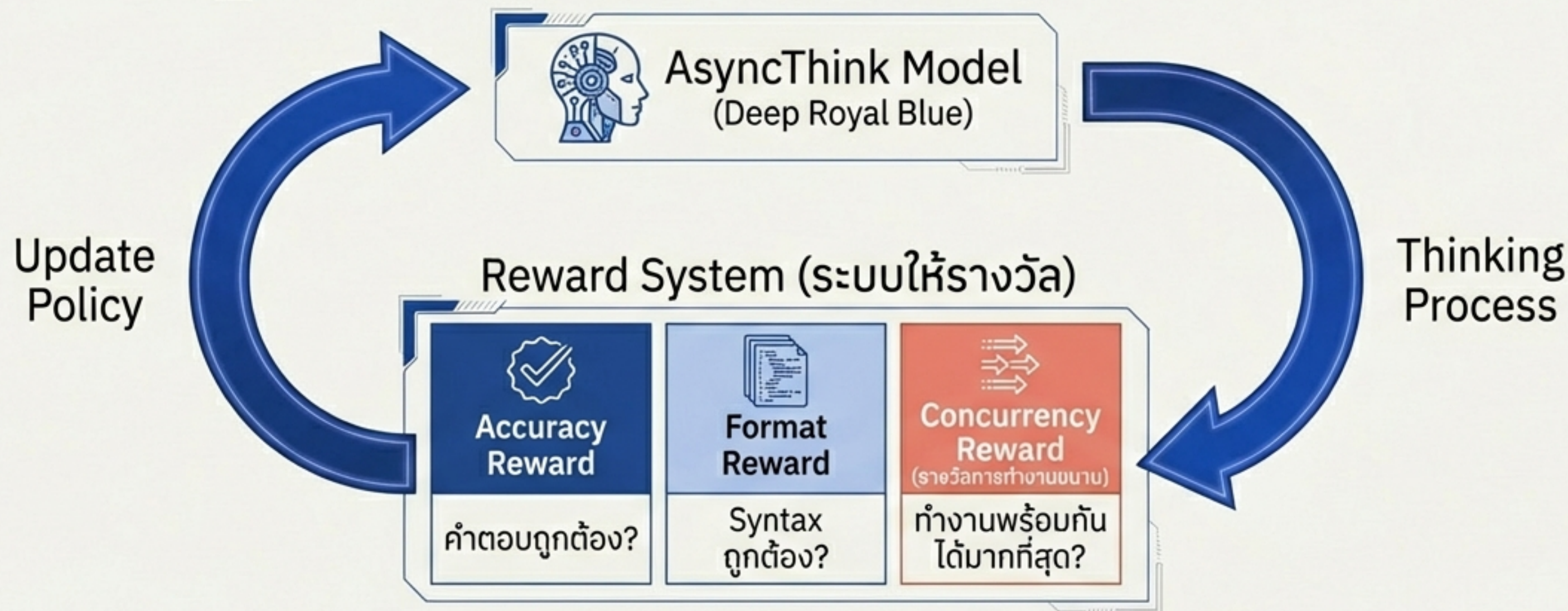


<FORK> ... <JOIN>
(Correct Syntax)

Problem: โมเดลเริ่มต้นไม่รู้จักวิธีการ Fork หรือ Join

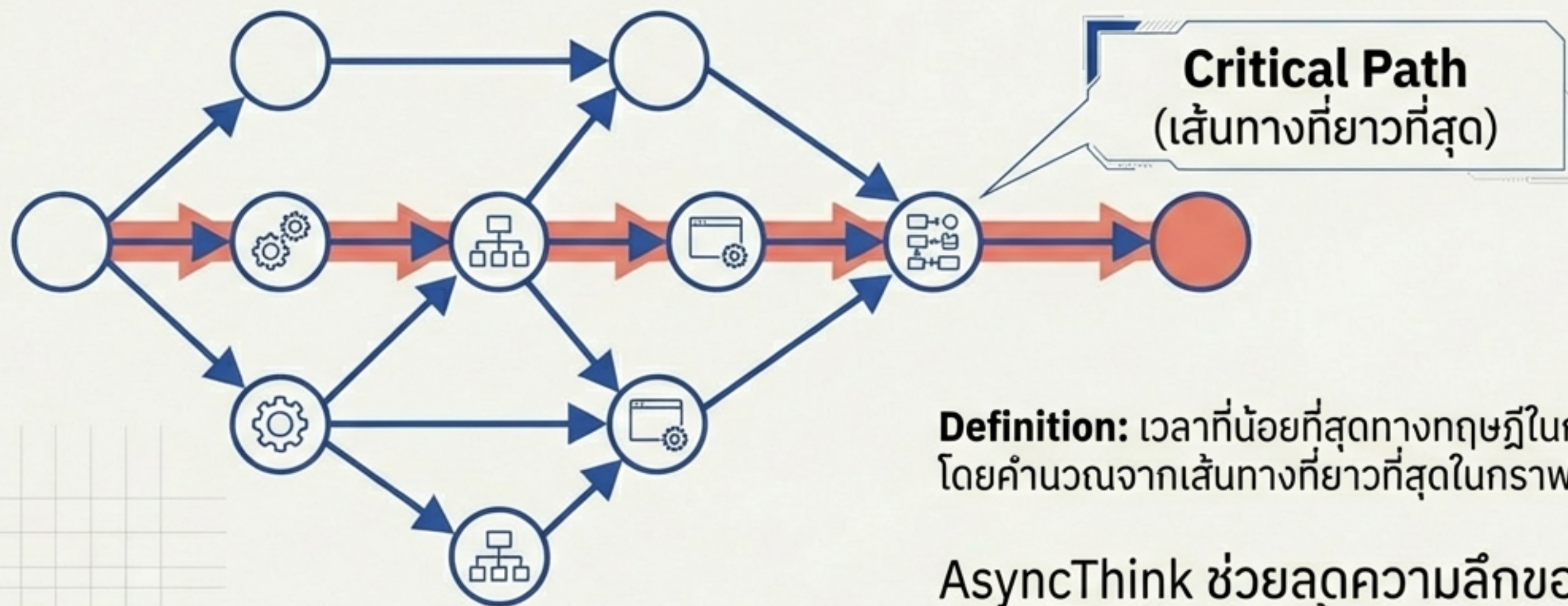
Solution: ใช้ข้อมูลสังเคราะห์สอนให้โมเดลจดจำ Syntax และบทบาทของ Organizer/Worker

ขั้นตอนที่ 2: ปรับปรุงด้วย Reinforcement Learning (RL)



Goal: ฝึกให้โมเดล 'โลก' ในการหาทางลัด (Efficiency) โดยไม่เสียความถูกต้อง (Accuracy)

การวัดผล: ความหน่วงของเส้นทางวิกฤต (Critical-Path Latency)

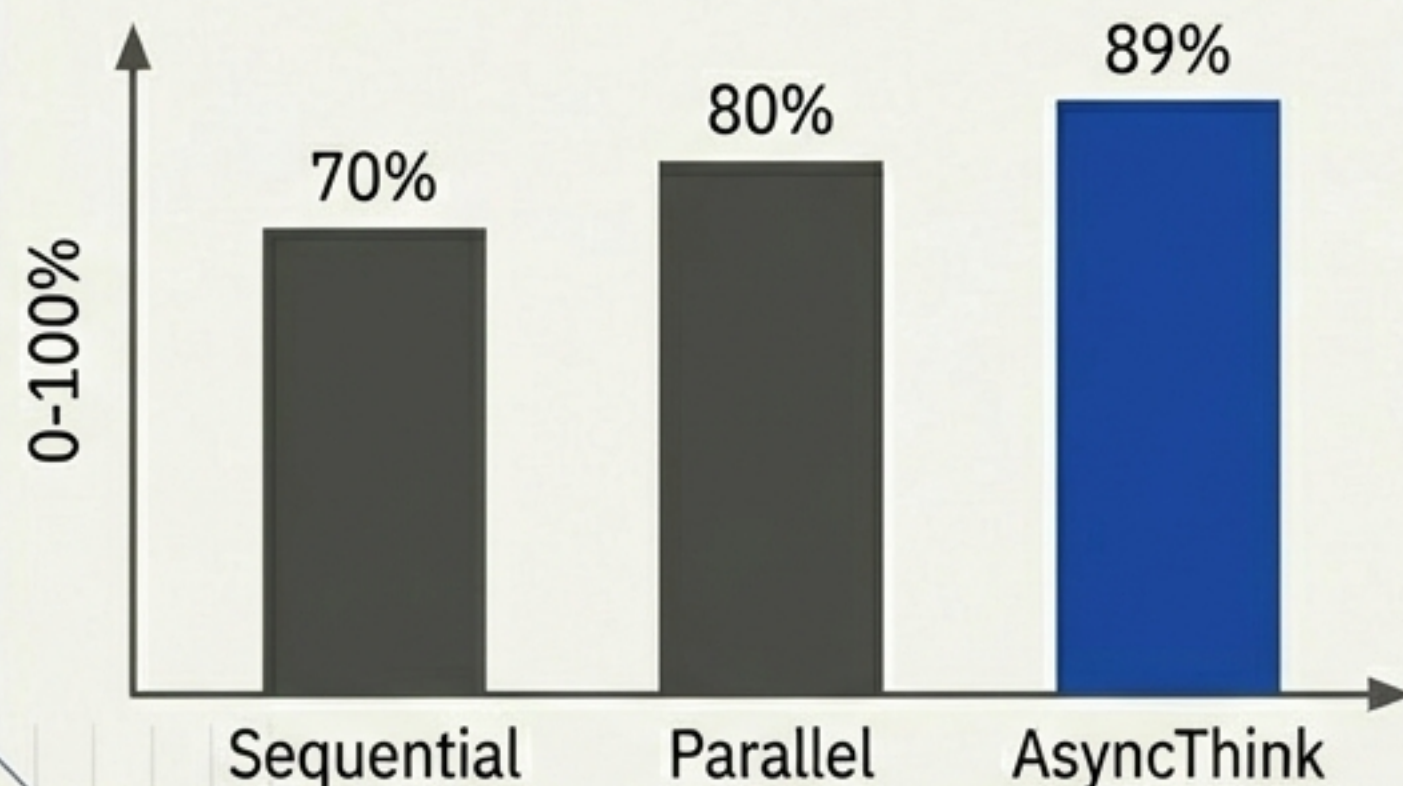


Definition: เวลาที่น้อยที่สุดทางทฤษฎีในการทำงานให้เสร็จสิ้น โดยคำนวณจากเส้นทางที่ยาวที่สุดในกราฟการทำงาน (DAG)

AsyncThink ช่วยลดความลึกของกราฟนี้ลง
ทำให้ได้คำตอบเร็วขึ้น

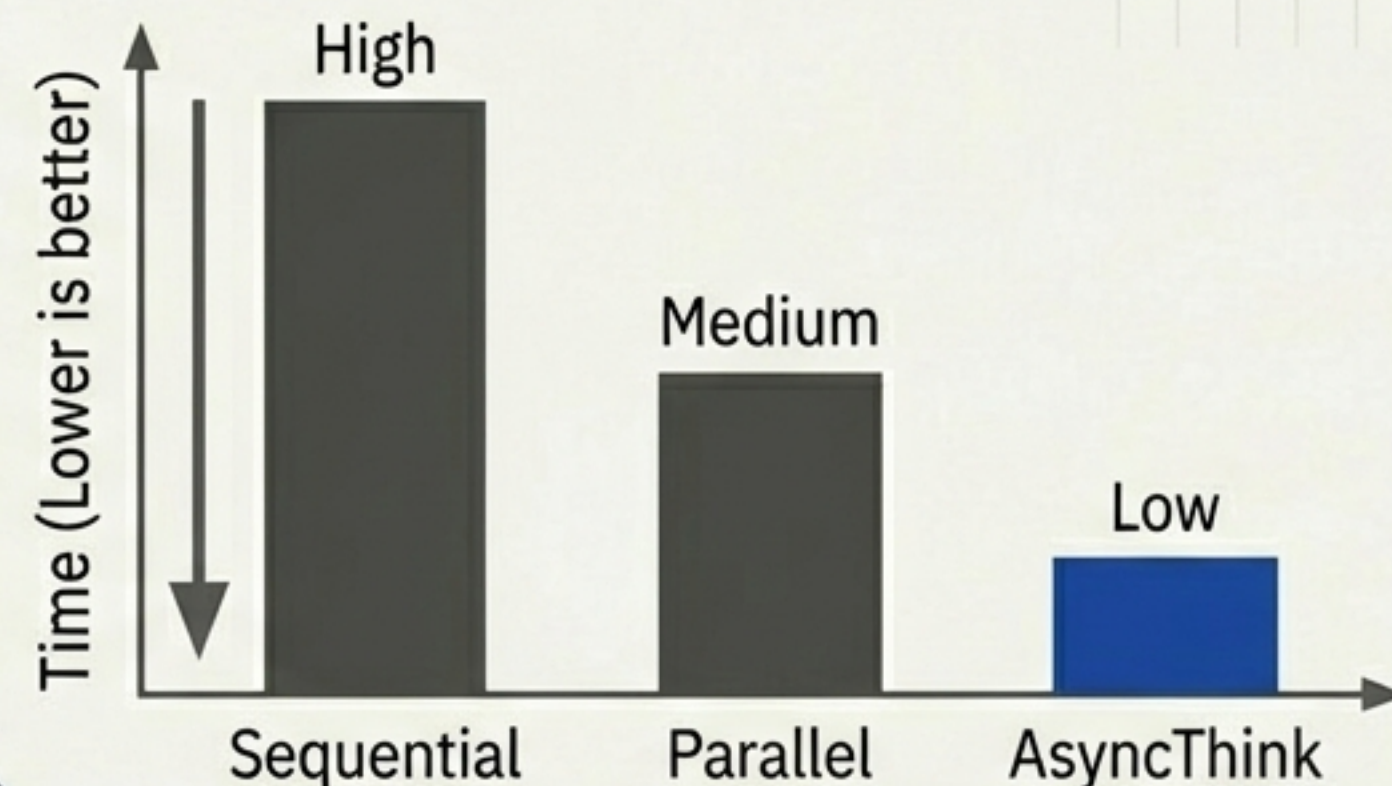
ผลลัพธ์: การให้เหตุผลทางคณิตศาสตร์

Accuracy (Countdown Task)



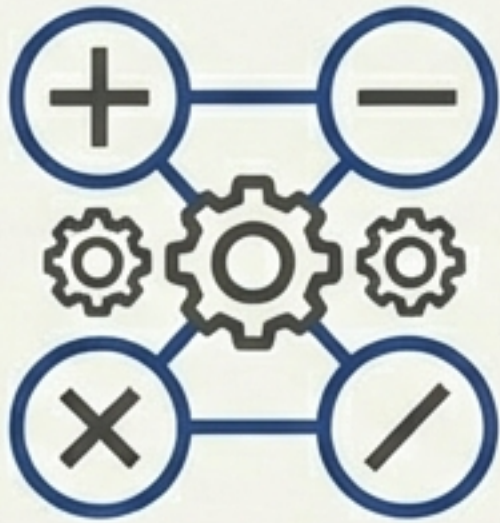
Countdown Accuracy:
89.0%

Latency (Math Reasoning)

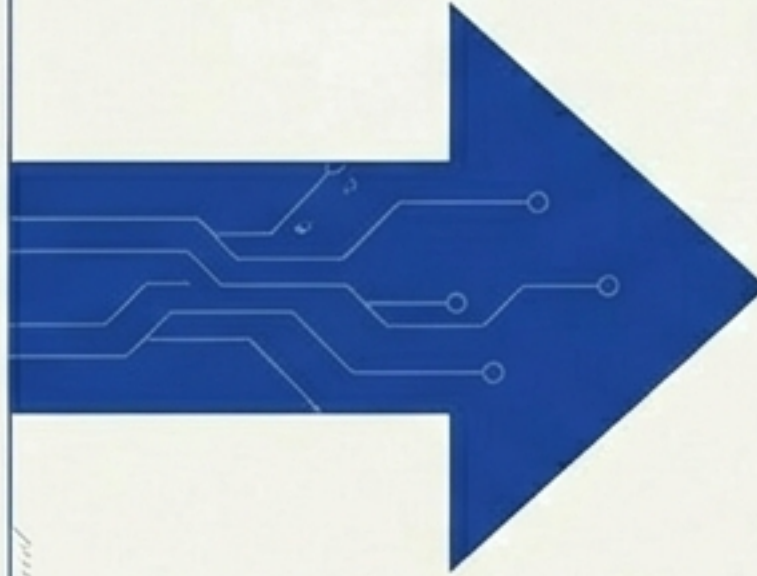


Superior
Latency & Accuracy

การเรียนรู้ข้ามสายงาน (Zero-Shot Generalization)



Training: Countdown



Testing: Sudoku

Result: โมเดลสามารถ **‘จัดระเบียบ’** การแก้โจทย์ **Sudoku** แบบ **Asynchronous** ได้เองโดยไม่ต้องเทรนเพิ่ม

Significance: โมเดลเรียนรู้นามธรรมของ **‘วิธีการแก้ปัญหา’ (Organization Policy)** ไม่ใช่แค่การจำคำตอบ

บทสรุป: อนาคตของระบบคิดแบบ AI

- ✓ **Dynamic Architecture:**
ปรับโครงสร้างการคิดตามความซับซ้อนของปัญหา (Fork/Join)
- ✓ **Self-Optimizing:**
เรียนรู้ที่จะทำงานให้เร็วขึ้นผ่าน Reinforcement Learning
- ✓ **Generalizable:**
ทักษะการจัดระเบียบสามารถนำไปใช้กับงานใหม่ๆ ได้ทันที

AsyncThink เปลี่ยน LLM จาก ‘ผู้คิดคนเดียว’ เป็น ‘ผู้จัดการทีม’

The Era of Agentic Organization

Zewen Chi, Li Dong, Qingxiu Dong, Yaru Hao,
Xun Wu, Shaohan Huang, Furu Wei



<https://aka.ms/GeneralAI>

Paper Reference: arXiv:2510.26658v1